



MS GLEE EDUCATION

www.msgleeducation.com
Data Engineering Training Program

DataFlow 360

End-to-End E-Commerce Analytics Pipeline

AWS Snowflake dbt Cloud Power BI

Complete Project Implementation Guide | Edition 2025-2026

Table of Contents

1	Project Overview	3
2	Tech Stack and Architecture	4
3	SDLC Model — Iterative Development	5
4	Dataset Description	7
STAGE 1 — AWS: Data Lake Setup		
5	S3 Bucket and File Upload	9
6	IAM User and Access Key Configuration	11
STAGE 2 — Snowflake: Data Warehouse		
7	Database, Schema and Warehouse Setup	13
8	External Stage and Data Loading	15
9	Data Manipulation Operations	17
10	Streams and Tasks: Change Data Capture	19
11	Views and Dynamic Tables	22
STAGE 3 — dbt Cloud: Transformation		
12	Account Setup and Project Configuration	25
13	Source and Staging Model Creation	27
14	dbt Operations and Testing	29
STAGE 4 — Power BI: Reporting		
15	Snowflake Connection Setup	32
16	Data Modelling and DAX Measures	33
17	Report Design and Dashboard	35
Quality Assurance and Testing		
18	Test Cases Across All Stages	37
19	Dev, Test and Production Lifecycle	41
20	Project Conclusion and CV Guide	43

1. Project Overview

DataFlow 360 is a complete end-to-end data engineering project designed to replicate a real-world production pipeline in the e-commerce domain. Built and delivered as part of MS GLEE Education's Data Engineering training program, this project takes students through the full journey of raw data — from storage and ingestion, through transformation and automation, to visual reporting.

1.1 What This Project Covers

The pipeline follows the modern data stack pattern used by data teams across the industry. Each stage uses a dedicated tool that serves a specific purpose:

Stage	Tool	Purpose
Stage 1	AWS S3	Data Lake — raw file storage and ingestion source
Stage 2	Snowflake	Data Warehouse — structured storage, computing, CDC and automation
Stage 3	dbt Cloud	Transformation Layer — SQL automation using the medallion architecture
Stage 4	Power BI	Reporting Layer — dashboards, data modelling and business insights

1.2 Business Scenario

Imagine an e-commerce company that processes thousands of orders every day. Their raw transactional data arrives as CSV files stored in cloud storage. The data engineering team must ingest raw files into a Data Lake, load and organise data into a Data Warehouse, automate cleaning and transformation, and deliver business dashboards for decision makers.

DataFlow 360 simulates exactly that workflow using four CSV datasets: Customers, Products, Orders and Order Items.

1.3 Learning Objectives

1. Understand the concept of a Data Lake versus a Data Warehouse
2. Configure AWS S3 and IAM access policies for secure programmatic access
3. Build Snowflake objects including databases, schemas, stages, tables, views, streams, tasks and dynamic tables
4. Use dbt Cloud to build a staging layer following the medallion architecture
5. Connect Power BI to Snowflake and build an interactive business dashboard
6. Apply SDLC practices including structured testing and environment promotion

2. Tech Stack and Architecture

2.1 Tools at a Glance

Tool	Role in This Project
AWS S3	Simple Storage Service — object storage used as the Data Lake landing zone for raw CSV files
AWS IAM	Identity and Access Management — controls who can access AWS resources; we create an IAM user with programmatic access
Snowflake	Cloud Data Warehouse supporting both storage and compute; handles SQL queries, transformations and automation
dbt Cloud	Data Build Tool — browser-based IDE that compiles SQL models into Snowflake transformations and promotes code reuse and testing
Power BI	Microsoft's business intelligence platform; connects directly to Snowflake to build interactive reports

2.2 End-to-End Data Flow

Data Flow Architecture	
[Raw CSV Files on Local Machine]	
v	
Stage 1: AWS S3 (Data Lake)	Raw files are stored here as objects
v	
Stage 2: Snowflake RAW Schema	Tables loaded from S3 via External Stage
(Streams and Tasks monitor changes automatically)	
v	
Stage 3: dbt Cloud Snowflake DEV Schema	Staging models written to
v	
Stage 4: Power BI Snowflake DEV	Reports and dashboards connect to

2.3 Medallion Architecture

This project follows the Medallion Architecture, an industry standard pattern for organising data into layers of increasing quality and structure.

Layer	Name	Snowflake Schema	What Happens Here
Bronze	Raw Layer	RAW	Data loaded as-is from S3. No transformations. Original data preserved exactly.
Silver	Staging Layer	DEV via dbt	Cleaned, renamed and lightly transformed using dbt staging models.
Gold	Analytics Layer	ANALYTICS (future)	Aggregated and business-ready tables consumed directly by Power BI.

3. SDLC Model — Iterative Development

3.1 What is SDLC?

The Software Development Life Cycle (SDLC) is a structured process used by IT teams to plan, design, build, test and deliver software and data systems. Following an SDLC model ensures every phase of a project is well-documented, testable and maintainable — which is exactly how professional data teams operate.

3.2 Why Iterative Development?

For DataFlow 360, the Iterative Development Model was chosen because each tool represents a self-contained iteration that produces a working, verifiable output. Students can validate each stage before moving forward, issues discovered in one iteration are resolved without restarting the project, and the approach mirrors how real data engineering teams work in sprints.

3.3 The Iterative Cycle Applied at Every Stage

Iterative Loop — Repeated at Each of the Four Stages	
PLAN	Define what needs to be built in this stage
DESIGN	Design the schema, model or report structure
DEVELOP	Write SQL, configure tools, build models
TEST	Run test cases to validate correctness
REVIEW	Check outputs and resolve any issues
PROCEED	Move to the next stage with verified outputs

3.4 SDLC Phases Mapped to DataFlow 360

SDLC Phase	Iteration / Stage	Activity in DataFlow 360
Requirements	All Stages	Define source datasets, destination systems, transformation needs and reporting KPIs
System Design	Stage 1 and 2	Design S3 folder structure, Snowflake schema layout, IAM permissions and external stage configuration
Development	Stage 2 and 3	Write DDL/DML in Snowflake, create streams, tasks, views and dynamic tables; build dbt staging models
Testing	All Stages	Run test cases per stage including row count checks, null validation, stream capture tests and dbt source freshness

Integration	Stage 3 and 4	dbt models write to the Snowflake DEV schema; Power BI connects and imports the staging tables
Deployment	Stage 4	Power BI report published and validated end-to-end across the complete pipeline
Maintenance	Ongoing	Tasks auto-refresh data, dbt runs on schedule and Power BI refreshes directly from Snowflake

3.5 Development, Testing and Production Environments

Every professional data team maintains three environments. DataFlow 360 simulates this three-tier model throughout the project.

Environment	Who Uses It	Snowflake Schema	Purpose
DEVELOPMENT	Student / Developer	DEV	Build and test new models here. Mistakes are safe and expected in this environment.
TESTING	Trainer / QA Team	TEST (simulated)	Validate data quality, run test cases and check business logic before going live.
PRODUCTION	Business Users	PROD (simulated)	Clean and verified data only. Power BI connects here in a real production scenario.

4. Dataset Description

DataFlow 360 uses four CSV files that simulate a real e-commerce company's transactional data. These files are intentionally designed with minor data quality issues to give students a realistic experience of identifying and resolving the kinds of problems encountered in actual data pipelines.

4.1 customers.csv (20 rows)

Column	Description
customer_id	Unique identifier for each customer, for example C001
name	Full name of the customer
email	Email address. Contains one duplicate entry as a deliberate data quality issue.
city	Customer's city of residence
country	Country name
signup_date	Date the customer registered, in YYYY-MM-DD format

4.2 products.csv (15 rows)

Column	Description
product_id	Unique identifier for each product, for example P001
product_name	Descriptive name of the product
category	Product category such as Electronics, Clothing or Home and Garden
price	Unit selling price in EUR
stock_quantity	Units currently available in inventory

4.3 orders.csv (75 rows)

Column	Description
order_id	Unique order identifier, for example O001
customer_id	Foreign key linking to customers.csv

order_date	Date the order was placed
status	Order status: Completed, Returned or Pending
total_amount	Total order value in EUR

4.4 order_items.csv (107 rows)

Column	Description
item_id	Unique line item identifier
order_id	Foreign key linking to orders.csv
product_id	Foreign key linking to products.csv
quantity	Units ordered. Contains one row with quantity equal to zero as a deliberate data quality issue.
unit_price	Price at the time of the order, which may differ from the current product price
subtotal	Calculated as quantity multiplied by unit_price

Intentional Data Quality Issues
Issue 1: One duplicate email address in customers.csv
Issue 2: One order_items row where quantity = 0
Issue 3: Two orders with status = Returned, testing filter logic Issue 4: Minor price differences between products.price and order_items.unit_price
These issues are deliberate. Students will detect and resolve them in the Snowflake and dbt stages.

4.5 Table Relationships

- customers and orders: one customer can have many orders
- orders and order_items: one order can have many line items
- products and order_items: one product can appear in many line items

STAGE 1**AWS — Data Lake Setup***Amazon S3 as the landing zone for raw CSV files*

5. S3 Bucket and File Upload

5.1 What is AWS S3?

Amazon S3 (Simple Storage Service) is an object storage service provided by AWS. In a modern data architecture, S3 acts as the Data Lake — the central landing zone where raw data arrives before any processing begins. Files are stored as objects inside containers called Buckets.

S3 Concept	Meaning
Bucket	A container for storing files in S3, similar to a top-level folder in the cloud
Object	Any file stored inside a bucket such as CSV, JSON or Parquet files
Prefix	A logical folder path inside a bucket, for example <code>/raw/orders/</code>
Region	The geographic AWS data centre where your bucket lives, for example eu-north-1 is Stockholm

5.2 Create Your S3 Bucket

7. Log in to the AWS Console at console.aws.amazon.com
8. In the search bar, type S3 and click the S3 service
9. Click Create Bucket
10. Enter a bucket name. Names must be globally unique across all of AWS. Use something like msglee-dataflow360.
11. Select your AWS Region. Use eu-north-1 (Stockholm) or your preferred region. Keep this consistent across all services.
12. Block Public Access: leave all settings checked ON. We use IAM credentials, not public access.
13. Click Create Bucket

5.3 Upload the CSV Files

14. Click on your newly created bucket to open it
15. Click Create Folder and name it raw — this is your Bronze layer landing folder
16. Click Upload and then Add Files
17. Select all four CSV files from your local machine
18. Click Upload and wait for the success confirmation
19. Verify: all four files should now appear in the S3 console with their file sizes visible

Expected S3 Structure After Upload

```
msglee-dataflow360/  
└─ raw/  
    ├── customers.csv  
    ├── products.csv  
    ├── orders.csv  
    └─ order_items.csv
```

6. IAM User and Access Key Configuration

6.1 What is AWS IAM?

IAM (Identity and Access Management) is AWS's security system. It controls which users and services can access which AWS resources and what actions they are allowed to perform. For this project, we create an IAM User with programmatic access so that Snowflake can connect to S3 using an Access Key and Secret Key.

Concept	Description
IAM User	A specific identity created for a person or application to access AWS services
IAM Policy	A JSON document defining what an IAM User is allowed and not allowed to do
Access Key ID	A public identifier used to authenticate — like a username for programmatic access
Secret Access Key	A private password paired with the Access Key ID. Never share this.
Programmatic Access	Access via code or tools using APIs rather than the AWS web console

6.2 Create the IAM User

20. In the AWS Console, search for IAM and open the IAM service
21. In the left sidebar, click Users and then click Add Users
22. Enter a username such as dataflow360-snowflake-user
23. Select Programmatic access as the access type — this generates an Access Key and Secret Key
24. Click Next: Permissions
25. Select Attach existing policies directly
26. Search for AmazonS3ReadOnlyAccess and tick the checkbox. This allows read access across S3.
27. Click Next through the remaining steps and then click Create User
28. On the success screen, click Download .csv to save your credentials. This is the only time AWS shows the Secret Access Key.

Credentials Needed for Snowflake Stage Setup	
Access Key ID:	e.g. AKIA*****MPLE
Secret Access Key:	e.g. wJalr*****EKEY

These will be entered into Snowflake when creating the External Stage.

Never commit these credentials to GitHub or share them in any public document.

STAGE 2**Snowflake — Data Warehouse***Organising, computing and automating the data*

7. Database, Schema and Warehouse Setup

7.1 Snowflake Architecture Overview

Snowflake separates Storage and Compute, meaning you pay for data storage and processing power independently. The platform operates across three layers: Cloud Services for authentication and query optimisation, the Compute Layer where Virtual Warehouses run SQL queries, and the Storage Layer where data is held in columnar compressed format on cloud storage.

7.2 Objects to Create

Object	Purpose
Database: dataflow360	The top-level container for all project objects
Schema: RAW	Holds tables loaded directly from S3. Bronze layer with untransformed data.
Schema: DEV	Development schema where dbt writes its staging models. Silver layer.
Virtual Warehouse: dataflow360_wh	X-Small size, auto-suspends after 60 seconds of inactivity
External Stage: aws_s3_stage	Pointer to the S3 bucket with IAM credentials stored securely

7.3 Create Database and Schemas

```
-- Create the Database
CREATE DATABASE dataflow360;

-- Use the new database
USE DATABASE dataflow360;

-- Create the RAW schema (Bronze Layer)
CREATE SCHEMA raw;

-- Create the DEV schema (Silver Layer / dbt target)
CREATE SCHEMA dev;
```

```
-- Verify both schemas were created
SHOW SCHEMAS IN DATABASE dataflow360;
```

7.4 Create the Virtual Warehouse

```
CREATE WAREHOUSE dataflow360_wh
  WAREHOUSE_SIZE = 'X-SMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

-- Use this warehouse for all subsequent queries
USE WAREHOUSE dataflow360_wh;
```

8. External Stage and Data Loading

8.1 What is an External Stage?

A Stage in Snowflake is a named reference to a file storage location. An External Stage points to a cloud storage location outside Snowflake — in our case the AWS S3 bucket. Once created, Snowflake can read files directly from S3 using the COPY INTO command.

8.2 Create the External Stage

```
USE SCHEMA dataflow360.raw;

CREATE OR REPLACE STAGE aws_s3_stage
  URL = 's3://msglee-dataflow360/raw/'
  CREDENTIALS = (
    AWS_KEY_ID      = 'YOUR_ACCESS_KEY_ID',
    AWS_SECRET_KEY = 'YOUR_SECRET_ACCESS_KEY'
  )
  FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY =
  '');

-- Verify files are accessible
LIST @aws_s3_stage;
```

8.3 Create the Raw Tables

```
-- CUSTOMERS
CREATE OR REPLACE TABLE raw.customers (
  customer_id  VARCHAR(10),
  name         VARCHAR(100),
  email       VARCHAR(100),
  city        VARCHAR(50),
  country     VARCHAR(50),
  signup_date  DATE
);

-- PRODUCTS
CREATE OR REPLACE TABLE raw.products (
  product_id   VARCHAR(10),
  product_name VARCHAR(150),
  category     VARCHAR(50),
  price        NUMBER(10,2),
```



```

    stock_quantity INT
);

-- ORDERS
CREATE OR REPLACE TABLE raw.orders (
    order_id      VARCHAR(10),
    customer_id   VARCHAR(10),
    order_date     DATE,
    status         VARCHAR(20),
    total_amount   NUMBER(10,2)
);

-- ORDER_ITEMS
CREATE OR REPLACE TABLE raw.order_items (
    item_id       VARCHAR(10),
    order_id       VARCHAR(10),
    product_id     VARCHAR(10),
    quantity       INT,
    unit_price     NUMBER(10,2),
    subtotal       NUMBER(10,2)
);

```

8.4 Load Data Using COPY INTO

```

COPY INTO raw.customers      FROM @aws_s3_stage/customers.csv;
COPY INTO raw.products      FROM @aws_s3_stage/products.csv;
COPY INTO raw.orders        FROM @aws_s3_stage/orders.csv;
COPY INTO raw.order_items   FROM @aws_s3_stage/order_items.csv;

-- Verify row counts
SELECT 'customers' AS table_name, COUNT(*) AS rows FROM raw.customers
UNION ALL
SELECT 'products',  COUNT(*) FROM raw.products
UNION ALL
SELECT 'orders',    COUNT(*) FROM raw.orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM raw.order_items;

```

Expected Row Counts After Load

customers	20 rows
-----------	---------

products	15 rows
orders	75 rows
order_items	107 rows

9. Data Manipulation Operations

9.1 DML Overview

Data Manipulation Language (DML) refers to SQL commands that change the data inside tables. In DataFlow 360, DML operations are practised on the RAW tables to build a clear understanding of how data is managed before automated pipelines take over.

Command	Purpose
INSERT	Add new rows to a table
UPDATE	Modify values in existing rows
DELETE	Remove specific rows from a table
MERGE	Upsert logic — insert if the record does not exist, update if it does. Core to CDC pipelines.
TRUNCATE	Remove all rows from a table without deleting the table structure itself

9.2 INSERT

```
INSERT INTO raw.orders (order_id, customer_id, order_date, status,
total_amount)
VALUES ('O076', 'C005', '2024-12-01', 'Completed', 149.99);
```

9.3 UPDATE

```
UPDATE raw.orders
SET    status = 'Completed'
WHERE  order_id = 'O076';
```

9.4 DELETE

```
-- Remove the order_item row where quantity is 0
DELETE FROM raw.order_items
WHERE  quantity = 0;
```

9.5 MERGE — Upsert Logic

```
MERGE INTO raw.customers AS target
USING (SELECT 'C005'           AS customer_id,
            'Updated Name'     AS name,
            'newemail@example.com' AS email) AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
    UPDATE SET target.name = source.name,
              target.email = source.email
WHEN NOT MATCHED THEN
    INSERT (customer_id, name, email)
    VALUES (source.customer_id, source.name, source.email);
```

10. Streams and Tasks — Change Data Capture

10.1 What is Change Data Capture?

Change Data Capture (CDC) is a technique that tracks INSERT, UPDATE and DELETE operations on a table so that downstream systems only process changed records rather than reprocessing everything. This is essential in production data pipelines to improve performance and efficiency.

10.2 Snowflake Streams

A Stream in Snowflake acts like a changelog attached to a table. It captures every DML change — INSERT, UPDATE or DELETE — that occurs after the stream was created. Each row in the stream includes a METADATA\$ACTION column indicating whether the change was an INSERT or a DELETE.

```
-- Create a stream on the orders table
CREATE OR REPLACE STREAM orders_stream
  ON TABLE raw.orders;

-- Insert a new order to trigger the stream
INSERT INTO raw.orders
VALUES ('O077', 'C010', '2024-12-05', 'Pending', 89.99);

-- Check what the stream captured
SELECT * FROM orders_stream;

-- METADATA$ACTION will show 'INSERT' for the new row
```

10.3 Snowflake Tasks

A Task is a scheduled SQL command in Snowflake, similar to a cron job but managed entirely within the platform. Tasks can be triggered on a time schedule or when a condition is met such as a stream containing data. In DataFlow 360, a Task automatically processes stream data into a target table.

```
-- Create a target table for processed orders
CREATE OR REPLACE TABLE raw.orders_processed LIKE raw.orders;

-- Create a task that processes the stream every minute
CREATE OR REPLACE TASK process_orders_task
  WAREHOUSE = dataflow360_wh
  SCHEDULE  = '1 MINUTE'
  WHEN
    SYSTEM$STREAM_HAS_DATA('orders_stream')
  AS
```

```
INSERT INTO raw.orders_processed
SELECT order_id, customer_id, order_date, status, total_amount
FROM   orders_stream
WHERE  METADATA$ACTION = 'INSERT';

-- Tasks are created in SUSPENDED state – activate it
ALTER TASK process_orders_task RESUME;

-- After 1 to 2 minutes, verify data was processed
SELECT * FROM raw.orders_processed;
```

How the Stream and Task Work Together

1. New data arrives in raw.orders via INSERT, UPDATE or DELETE
2. The stream captures the change automatically
3. SYSTEM\$STREAM_HAS_DATA() returns TRUE
4. The task wakes up and inserts the change into orders_processed
5. The stream offset moves forward – the change is consumed

This is exactly how real production CDC pipelines operate.

11. Views and Dynamic Tables

11.1 Views in Snowflake

A View is a saved SQL query that behaves like a virtual table. It does not store data — it always executes the underlying query when accessed. Views are used to hide sensitive columns, simplify complex queries into reusable objects, and provide a stable interface even when the underlying table structure changes.

```
-- View 1: Hide email from analysts
CREATE OR REPLACE VIEW dev.vw_customers_public AS
SELECT customer_id, name, city, country, signup_date
FROM   raw.customers;

-- View 2: Show only completed orders
CREATE OR REPLACE VIEW dev.vw_completed_orders AS
SELECT * FROM raw.orders WHERE status = 'Completed';

-- View 3: Revenue summary by product category
CREATE OR REPLACE VIEW dev.vw_revenue_by_category AS
SELECT   p.category,
          SUM(oi.subtotal)           AS total_revenue,
          COUNT(DISTINCT o.order_id) AS total_orders
FROM     raw.order_items oi
JOIN     raw.orders      o   ON oi.order_id = o.order_id
JOIN     raw.products    p   ON oi.product_id = p.product_id
WHERE    o.status = 'Completed'
GROUP BY p.category;
```

11.2 Secure Views

A Secure View hides the SQL definition from other users. In a production environment this ensures analysts cannot reverse-engineer business logic or access sensitive join conditions.

```
CREATE OR REPLACE SECURE VIEW dev.vw_order_summary AS
SELECT o.order_id,
       c.name           AS customer_name,
       o.order_date,
       o.status,
       o.total_amount
FROM   raw.orders      o
JOIN   raw.customers c ON o.customer_id = c.customer_id;
```

11.3 Dynamic Tables

A Dynamic Table is a Snowflake-native materialised view that automatically refreshes when source data changes. Unlike a regular view which queries on every access, a Dynamic Table stores pre-computed results and refreshes on a scheduled target lag. This is ideal for aggregations consumed by Power BI.

```
CREATE OR REPLACE DYNAMIC TABLE dev.dt_daily_revenue
  TARGET_LAG = '1 minute'
  WAREHOUSE = dataflow360_wh
AS
SELECT
  DATE_TRUNC('day', o.order_date) AS order_day,
  p.category,
  SUM(oi.subtotal)                AS daily_revenue,
  COUNT(DISTINCT o.order_id)      AS order_count
FROM   raw.orders      o
JOIN   raw.order_items oi ON o.order_id = oi.order_id
JOIN   raw.products    p  ON oi.product_id = p.product_id
WHERE  o.status = 'Completed'
GROUP BY 1, 2;

SELECT * FROM dev.dt_daily_revenue ORDER BY order_day DESC;
```


STAGE 3**dbt Cloud — Data Transformation**
Automating SQL with version-controlled, testable models

12. dbt Cloud — Account Setup and Project Configuration

12.1 What is dbt?

dbt (Data Build Tool) is a transformation framework that allows data teams to write modular SQL queries called models, which are compiled and executed against the data warehouse. dbt Cloud is the browser-based version — no local installation is required, making it ideal for training environments.

dbt Concept	Meaning
Model	A .sql file containing a SELECT query. dbt compiles it into a table or view in Snowflake.
Source	A reference to an existing raw table in Snowflake, declared in a YAML configuration file.
ref()	dbt's function for referencing another model, enabling automatic dependency resolution between models.
Profile	Configuration linking dbt to your Snowflake connection details.
Target	The Snowflake schema where dbt writes its output, for example the DEV schema.

12.2 Account Setup

29. Go to cloud.getdbt.com and create a free Developer account
30. Create a new project and name it DataFlow 360
31. Select Snowflake as the database connection type
32. Enter your Snowflake connection details: account identifier, database name (dataflow360), warehouse (dataflow360_wh), target schema (dev), and your username and password
33. For the repository, select Managed Repository — the simplest option for a training environment
34. Click Test Connection and confirm you see a green success message

12.3 Development Environment Setup

35. In dbt Cloud go to Account Settings, then Projects, then DataFlow 360
36. Click Environments and then New Environment
37. Name: Development, Type: Development, Target Schema: dev
38. Enter your Snowflake credentials and save the environment
39. Go to Develop and open the dbt Studio IDE

13. Source and Staging Model Creation

13.1 Recommended Folder Structure

```
models/
├── sources.yml           declares raw Snowflake tables as dbt sources
└── staging/
    ├── stg_customers.sql
    ├── stg_products.sql
    ├── stg_orders.sql
    └── stg_order_items.sql
```

13.2 sources.yml

```
version: 2

sources:
  - name: raw
    database: dataflow360
    schema: raw
    tables:
      - name: customers
      - name: products
      - name: orders
      - name: order_items
```

13.3 Staging Models

Staging models apply light cleaning — renaming columns, casting data types and filtering out invalid records. These are your Silver layer outputs.

```
-- stg_customers.sql
SELECT
  customer_id,
  INITCAP(name)    AS customer_name,
  LOWER(email)     AS email,
  city,
  country,
  signup_date
FROM {{ source('raw', 'customers') }}
```

```
-- stg_orders.sql
SELECT
  order_id,
  customer_id,
  order_date,
  UPPER(status) AS order_status,
  total_amount
FROM {{ source('raw', 'orders') }}
WHERE status != 'Returned'
```

```
-- stg_order_items.sql
SELECT
  item_id,
  order_id,
  product_id,
  quantity,
  unit_price,
  subtotal
FROM {{ source('raw', 'order_items') }}
WHERE quantity > 0
```

14. dbt Operations and Testing

14.1 Running Models

dbt run	runs all models
dbt run --select staging.*	runs only the staging models
dbt run --select stg_orders	runs a single named model

14.2 Built-in Tests

```
version: 2
models:
  - name: stg_customers
    columns:
      - name: customer_id
```

```
    tests:
      - unique
      - not_null
    - name: email
      tests:
        - not_null

-- Run all tests
dbt test
```

14.3 Useful dbt Commands

Command	Purpose
dbt debug	Checks the Snowflake connection and verifies all configuration settings
dbt compile	Compiles SQL without executing — useful for reviewing the generated SQL before running
dbt source freshness	Checks whether source tables have been updated recently
dbt docs generate	Generates a documentation website from your models and YAML files

STAGE 4**Power BI — Reporting and Visualisation***Transforming clean data into business insights*

15. Snowflake Connection Setup in Power BI

15.1 Connection Details

Parameter	Value
Server	YOUR_ACCOUNT.snowflakecomputing.com
Warehouse	dataflow360_wh
Database	dataflow360
Schema	dev
Authentication	Username and Password — your Snowflake login credentials

15.2 Connection Steps

40. Open Power BI Desktop
41. Click Get Data and search for Snowflake, then select it
42. Enter the Server and Warehouse values from the table above
43. Enter your Snowflake username and password when prompted
44. In the Navigator window, expand dataflow360 and then dev
45. Select all four staging tables: stg_customers, stg_products, stg_orders and stg_order_items
46. Click Load to import the tables into your Power BI model

16. Data Modelling and DAX Measures

16.1 Define Relationships in Model View

From	To	Cardinality	Direction
stg_orders.customer_id	stg_customers.customer_id	Many to One	Single
stg_order_items.order_id	stg_orders.order_id	Many to One	Single
stg_order_items.product_id	stg_products.product_id	Many to One	Single

16.2 DAX Measures

Total Revenue	= SUM(stg_order_items[subtotal])
Total Orders	= DISTINCTCOUNT(stg_orders[order_id])
Avg Order Value	= DIVIDE([Total Revenue], [Total Orders])
Revenue by Cat	= CALCULATE([Total Revenue], RELATEDTABLE(stg_products))

17. Report Design and Dashboard

17.1 Recommended Visuals

Visual Type	What to Show
KPI Cards	Total Revenue, Total Orders, Average Order Value, Total Customers
Bar Chart	Revenue broken down by Product Category
Line Chart	Daily or monthly revenue trend over time
Table Visual	Top 10 customers ranked by total spend
Donut Chart	Order status distribution — Completed versus Pending
Map Visual	Customer distribution by city or country

17.2 Suggested Report Pages

- 47. Executive Summary — KPI cards, revenue trend and top categories
- 48. Order Analysis — order status breakdown and average order value by month
- 49. Product Performance — top products, revenue by category and stock versus sales comparison
- 50. Customer Insights — signup trends, geographic distribution and top customers by spend

18. Test Cases Across All Stages

Testing is a core part of the SDLC. The following test cases ensure that each stage of the DataFlow 360 pipeline produces correct and reliable output. In a professional IT environment these would typically be tracked in a test management tool such as Jira or Azure DevOps.

18.1 Stage 1 — AWS

Test ID	Test Case	Expected Result	Status
TC-AWS-01	S3 bucket created successfully	Bucket is visible in the AWS S3 console	Pass / Fail
TC-AWS-02	All 4 CSV files uploaded	LIST @stage returns 4 files with correct sizes	Pass / Fail
TC-AWS-03	IAM user has correct policy	Snowflake can read S3 without a permissions error	Pass / Fail
TC-AWS-04	Access Key credentials valid	COPY INTO completes without an authentication error	Pass / Fail

18.2 Stage 2 — Snowflake

Test ID	Test Case	Expected Result	Status
TC-SF-01	Database and schemas created	SHOW SCHEMAS returns RAW and DEV	Pass / Fail
TC-SF-02	Row counts after COPY INTO	20, 15, 75 and 107 rows in respective tables	Pass / Fail
TC-SF-03	INSERT new order	Row O077 is visible in the orders table	Pass / Fail
TC-SF-04	DELETE zero-quantity row	No rows exist where quantity = 0 in order_items	Pass / Fail
TC-SF-05	Stream captures INSERT	orders_stream shows the new row with METADATA\$ACTION = INSERT	Pass / Fail
TC-SF-06	Task processes stream data	orders_processed contains the captured row within 2 minutes	Pass / Fail
TC-SF-07	View hides email column	vw_customers_public does not include the email column	Pass / Fail
TC-SF-08	Dynamic Table refreshes	dt_daily_revenue updates after a new completed order is inserted	Pass / Fail

18.3 Stage 3 — dbt

Test ID	Test Case	Expected Result	Status
TC-DBT-01	dbt connection test passes	dbt debug returns OK for all checks	Pass / Fail
TC-DBT-02	All models run successfully	All 4 staging models show green in the dbt run output	Pass / Fail
TC-DBT-03	Returned orders filtered	stg_orders row count is less than 75	Pass / Fail
TC-DBT-04	Zero-quantity rows removed	stg_order_items row count is less than 107	Pass / Fail
TC-DBT-05	Uniqueness test passes	dbt test on customer_id unique returns no failures	Pass / Fail
TC-DBT-06	Not-null test passes	dbt test on order_id not_null returns no failures	Pass / Fail

18.4 Stage 4 — Power BI

Test ID	Test Case	Expected Result	Status
TC-PBI-01	Snowflake connection works	Tables load without error in the Power BI Navigator	Pass / Fail
TC-PBI-02	Relationships are defined	No inactive or broken relationships appear in Model View	Pass / Fail
TC-PBI-03	Total Revenue measure	Value equals the sum of all completed order_items subtotals	Pass / Fail
TC-PBI-04	Category slicer filter works	Selecting Electronics filters all visuals correctly	Pass / Fail
TC-PBI-05	All report pages render	All 4 report pages load without errors	Pass / Fail

19. Development, Testing and Production Lifecycle

In professional data engineering every change passes through three environments before reaching end users. This section describes how DataFlow 360 simulates this lifecycle.

19.1 Environment Promotion Flow



19.2 Simulating the Lifecycle in This Project

Environment	Activity
DEV Schema	All SQL and dbt models are written and tested here. Mistakes are expected and safe in this environment.
TEST (Simulated)	Copy staging model outputs to a TEST schema and run the Section 18 test cases to validate before promotion.
PROD (Simulated)	Once all tests pass, copy verified tables to a PROD schema. This is where Power BI would connect in a real scenario.

19.3 Simulating Promotion in Snowflake

```
-- Create the PROD schema
CREATE SCHEMA IF NOT EXISTS dataflow360.prod;

-- Promote a staging table to PROD after all tests pass
CREATE OR REPLACE TABLE prod.orders AS
SELECT * FROM dev.stg_orders;

-- In a real scenario, Power BI connects to the prod schema
```

19.4 Best Practices

- Never write directly to PROD — always develop in DEV first
- All changes should be version-controlled — dbt manages this automatically via its managed repository
- Test cases must pass in TEST before promotion to PROD
- Document every change using comments in SQL and model descriptions in dbt
- Auto-suspend your Snowflake warehouse to avoid unnecessary costs

20. Project Conclusion, Outcomes and CV Guide

20.1 What Was Built

DataFlow 360 is a complete, production-style data engineering pipeline covering the full journey from raw CSV files to interactive business dashboards. It demonstrates every layer of the modern data stack using industry-standard tools that are actively used by data teams across global organisations.

20.2 Key Learning Outcomes

Skill Area	What You Can Now Do
Cloud Storage	Create S3 buckets, upload files and understand object storage as a Data Lake
IAM Security	Create IAM users, attach policies and generate access keys for programmatic access
Snowflake	Build databases, schemas, external stages, raw tables, streams, tasks, views and dynamic tables
Change Data Capture	Implement CDC using Streams and Tasks for automated real-time data processing
dbt Transformation	Build modular SQL models following the medallion architecture with built-in testing
Power BI Reporting	Connect to Snowflake, model data relationships, write DAX measures and build dashboards
SDLC and Testing	Apply iterative development, write structured test cases and simulate Dev/Test/Prod promotion

20.3 Adding This Project to Your CV

Suggested CV Project Entry
DataFlow 360 – End-to-End E-Commerce Analytics Pipeline
Technologies: AWS S3, AWS IAM, Snowflake, dbt Cloud, Power BI
Duration: [Your duration here] Delivered via: MS GLEE Education – msgleeducation.com Built a complete data engineering pipeline simulating a production e-commerce analytics system from raw file ingestion to executive dashboards.

20.4 Roles and Responsibilities to List

Role / Title	CV Bullet Points
Data Engineer	- Designed and implemented an end-to-end pipeline using AWS S3, Snowflake, dbt Cloud and Power BI - Configured IAM policies and S3 external stages for secure programmatic access - Implemented CDC pipelines using Snowflake Streams and Tasks - Built dbt staging models following the medallion architecture (Bronze, Silver, Gold)
Cloud Data Engineer	- Provisioned and managed cloud infrastructure on AWS (S3, IAM) and Snowflake - Automated data transformation workflows using dbt Cloud and SQL - Connected Power BI to Snowflake for real-time business reporting
BI Developer	- Developed Power BI dashboards connected to Snowflake Data Warehouse - Created DAX measures for revenue, order volume and customer KPIs - Modelled data relationships following a star schema pattern
Data Analyst	- Analysed e-commerce transaction data across four datasets - Built SQL views and dynamic tables in Snowflake for business reporting - Identified and resolved data quality issues including duplicates and invalid entries

20.5 ATS Keywords to Include

Category	Keywords to Use
Cloud	Amazon Web Services, AWS S3, AWS IAM, Cloud Data Lake, Object Storage
Data Warehouse	Snowflake, Virtual Warehouse, External Stage, COPY INTO, Time Travel
Automation	Snowflake Streams, Snowflake Tasks, CDC, Change Data Capture
Transformation	dbt, dbt Cloud, SQL Models, Medallion Architecture, Bronze Silver Gold
Reporting	Power BI, DAX, Power Query, Data Visualisation, Dashboard Development, Star Schema
Process	Iterative Development, SDLC, Test Cases, UAT, Dev Test Production, Data Quality



MS GLEE EDUCATION

Bridging the gap between what you learned and what industry needs.

www.msgleeducation.com